

Most of the time we consider deep learning as an engineering problem – we are told what works what doesn't, what standard solution is to a specific problem. But very few people ask why. Geometric Deep Learning provides us with the mathematical tools to understand why all those famous architectures, including CNN, LSTM, Transformer etc. work better than plain MLP.

To further explain how it works and why it is useful, we will skip all the math in the front part of [Original Geometric Deep Learning book](#) and give you a quick walk-through to derive CNN from shift-equivariance.

💡 Tip

Before you start

Here are some nice youtube videos can give some sense of Group Theory. It's not required to understand this article, but it definitely helps.

- [Group Theory Step-by-Step: 1 - 7](#) by TheGrayCuber
- [What is Group Theory? – Group Theory Ep. 1](#) by Nemean

No need to remember all the details, feel it.

Shift Equivariance

Let's start with an 1D example with 4 pixels.

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

You can imagine, each value of x is the brightness of a pixel where $x \in [0, 1]$.

Since we want to discuss shift equivariance, we need to first define what is a shift operation. To make things simple at first, we will assume images with no boundary, which means positions wrap around in a circle. For example, if we shift the image by 1 pixel to the right, the new image shall be:

$$\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} \rightarrow \begin{pmatrix} x_3 \\ x_0 \\ x_1 \\ x_2 \end{pmatrix}$$

Let's call this shift operation S . Naturally we got $S \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} x_3 \\ x_0 \\ x_1 \\ x_2 \end{pmatrix}$.

i Note

If you were familiar with group theory, you should notice we've already formed a group $G = \{I, S, S^2, S^3\}$. But I want to make some obvious things clear for those who have zero background in group theory.

- An operation can composite multiple times, and we denote that as S^n , n is the number of times we apply the operation.

$$S^2 \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} x_2 \\ x_3 \\ x_0 \\ x_1 \end{pmatrix}$$

Now suppose we have an image-processing layer F that takes an image as input and output a feature map with the same size.

$$W \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix}$$

Then the shift equivariance is simply $F(S(x)) = S(F(x))$, meaning **the shift in the input will result in the same shift in the output**.

Sometimes we call properties like this **Inductive Bias**. To my understanding, Inductive Bias are some kinds of rules(Or prior knowledge in fancy words) we observed from our data. For example, in the model's perspective, Shift Equivariance means that if the cat in the image moved a few pixels, the model should still be able to recognize that's a cat. By embedding those rules into loss function(e.g. by adding regularization) or architecture, we can make the model work more efficiently. We will see exactly how that works in the next chapter.

Deriving CNN

CNN is widely used in CV tasks and label matching problems. To get started, we will focus on the image in one dimension, i.e. a sequence of pixels. To represent it, we denote image signal x as a function on domain $\Omega = (0, \dots, n-1)$. Hence, $x_u = x(u)$ where $u \in \Omega$ is a index of the sequence is the britness of the u^{th} pixel. The full input image is written as

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \\ \dots \\ x_{n-1} \end{pmatrix}.$$

At first gance, its a bit akward to define an image in such a perplexing way rather than a simple column vector directly. However, this definition is rather powerful as a language to represent signal for any other netwrok architectures, whiches will be discussed later. A good way to think about it is to imagie domain Ω as the medium where data live in. For an one dimensional image, the space the data live in is simply an ordered discrete grid with index. Whereas for an graph, the space where the nodes live in is an unordered discrete set $\{0, \dots, n-1\}$.

Suppose we are dealing with an image classification or feature extraction task where the core geometric structure of the data is spatial translation. If we shift a signal x to the right by a units, we obtain the new signal $S_a x$. Its value at index u is given by:

$$(S_a x)_u = x_{(u-a) \bmod n}$$

On a discrete grid (with periodic boundary conditions), the unit shift S_1 corresponds to a simple permutation matrix:

$$S_1 = \begin{bmatrix} 0 & 0 & \dots & 0 & 1 \\ 1 & 0 & \dots & 0 & 0 \\ 0 & 1 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & 0 \end{bmatrix}$$

Suppose we want to construct a neural network layer $F(x) = Wx$, where $W \in \mathbb{R}^{n \times n}$ is the weight matrix to be determined). We require this layer to satisfy shift equivariance. In operator form, this means that matrix multiplication by W and the shift operator S_1 must commute

$$W(S_1 x) = S_1(Wx) \Rightarrow WS_1 = S_1 W$$

Using elementary linear algebra, let us examine the restrictions that this commutativity constraint $WS_1 = S_1 W$ imposes on the entries $W_{u,v}$ of matrix W . Right-multiplication by the shift operator $(WS_1)_{u,v}$ shifts column $v + 1$ of W to column v , which yields

$$(WS_1)_{u,v} = W_{u,(v+1) \bmod n}$$

.

Whereas the left-multiplication by the shift operator, $(S_1 W)_{u,v}$ shifts row $u - 1$ of W to row u , which yields

$$(S_1 W)_{u,v} = W_{(u-1) \bmod n, v}$$

From $WS_1 = S_1 W$, we obtain the fundamental constraint equation:

$$W_{u,v+1 \bmod n} = W_{(u-1) \bmod n, v} \quad \forall u, v$$

letting $u' = u - 1$,

$$W_{(u'+1) \bmod n, (v+1) \bmod n} = W_{u', v} \quad \forall u', v$$

This relations reveals that a shift equivariant matrix must have the same value along its main diagonal and entries parallel to it (with periodic boundary conditions). Setting $k = (u - v) \bmod n$, this demonstrates that the entry $W_{u,v}$ depends solely on the index difference $(u - v)$. If we define an n -dimensional parameter vector $\theta_k := W_{k,0}$, then

$$W_{u,v} = \theta_{(u-v) \bmod n}$$

Substituting this back into W , its structure is necessarily

$$W = C(\theta) = \begin{bmatrix} \theta_0 & \theta_{n-1} & \theta_{n-2} & \dots & \theta_1 \\ \theta_1 & \theta_0 & \theta_{n-1} & \dots & \theta_2 \\ \theta_2 & \theta_1 & \theta_0 & \dots & \theta_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \theta_{n-1} & \theta_{n-2} & \theta_{n-3} & \dots & \theta_0 \end{bmatrix}$$

Now, consider the u -th component of the feature map $y = Wx = C(\theta)x$:

$$y_u = \sum_{v=0}^{n-1} W_{u,v} x_v = \sum_{v=0}^{n-1} \theta_{(u-v) \bmod n} x_v = (x * \theta)_u$$

This is precisely the definition of discrete convolution. In other words, the mathematical framework naturally deduces the convolution formula from linearity and shift equivariance